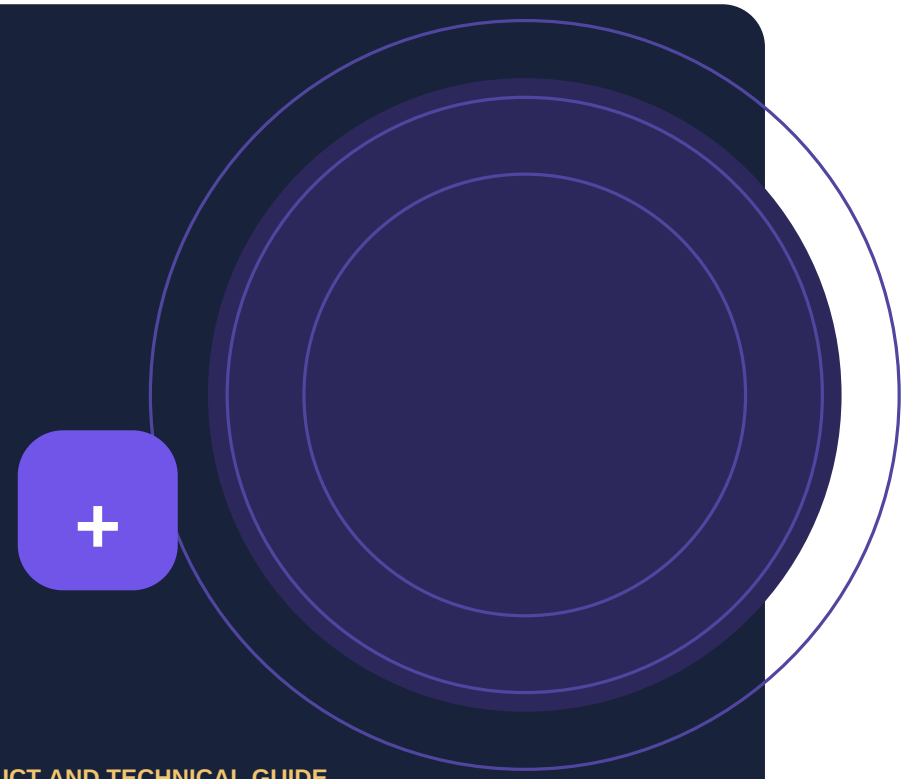


THE PRODUCT AND TECHNICAL GUIDE

StudyQuest

How the learning platform works and how it is built

A clear guide to learner features, administration, AI and deployment

Learn with purpose. Grow with every quest.

Prepared October 2026

How to use this guide

This document describes StudyQuest as implemented in the repository in October 2026. It explains what learners can do, how administrators operate the platform, how the main parts fit together, and what is needed to run the app.

> **What StudyQuest is**
> A Django learning platform that turns a topic or study notes into a structured, level-based quiz quest. Learners practice, receive feedback, collect XP and badges, and earn certificates when they complete all levels.

At a glance

Area	What it does
Public site	Landing page, configurable hero slides, product information, contact form and public certificate verification.
Learner workspace	Dashboard, quests, learning attempts, profile, notifications, leaderboard and certificates.
Platform management	User and role management, site identity, integrations, hero slides and contact inbox.
Core stack	Python and Django 5.2, server-rendered templates, HTML/CSS/JavaScript and relational data.
Connected services	Gemini/Gemma and Groq for AI, TiDB Cloud for production data, Cloudinary for media, SMTP for email, Vercel for hosting.

A feature may be implemented in the app while still needing a valid provider key, database, email account or deployment setting to work in production. This guide distinguishes those two parts. No secret keys or passwords are included.

1. Roles and access

StudyQuest has a public experience, an authenticated learner experience and staff administration. Django views enforce access on the server.

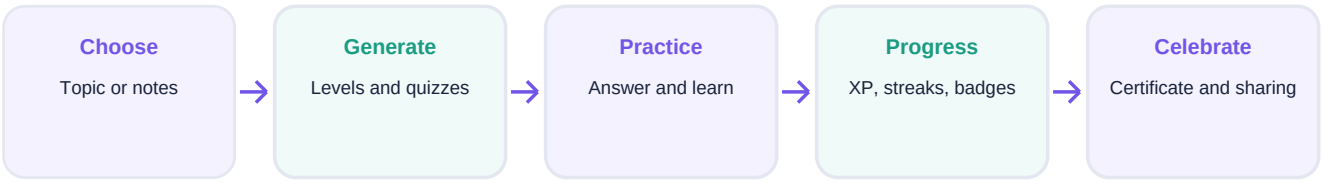
Role	Typical access
Visitor	Browse the home page and public information, submit a contact message, view public certificate verification and register when registration is enabled.
Learner	Manage their profile and quests, answer levels, view results and notifications, invite ordinary learners, see the leaderboard and share certificates.
Staff administrator	Use the management area to review users, contact messages, hero slides and site settings. Staff-only routes require an authenticated staff account.
Platform owner	The superuser can manage users and roles, set passwords, update integrations and site settings, and perform owner-only actions.

Account identity
Email is unique and normalized to lowercase. Sign-in uses email and password. Django hashes passwords. The admin form hashes a new password instead of storing readable text.

Any learner can invite an ordinary learner. Only the superuser can select an elevated invitation role or change system roles. Invitation links expire after seven days and cannot be reused after acceptance.

2. The learner journey

The main path takes a learner from a topic to a completed quest and a verifiable credential.



After signing in, the learner reaches a dashboard with current quests, recent attempts, activity, accuracy, badges and certificate count. Responsive navigation keeps workspace links available. A learner starts a quest by entering a topic, adding context, choosing difficulty and selecting the number of levels.

Study material can be typed, provided as a URL, or uploaded as a text file or PDF. The app extracts available text and sends a bounded amount as context to the generation service. The quest list supports search, status and difficulty filters, and sorting. A learner can open, continue or delete their own quest and its learning records.

> **Progression rule**
> Levels unlock in order. A later level becomes available after a passing attempt on the previous level. Each level stores its own pass threshold; 70 percent is the default.

3. AI quests and learning mechanics

AI builds structured learning material and can explain answers. Generated content is checked before it becomes a playable quest.

Quest generation first calls Gemini with the configured Gemma model. If generation fails, the app can fall back to Groq using the configured GPT-OSS-20B model. The response is requested as JSON and validated for the expected number of levels, at least three questions per level, four choices per question and valid answer indexes.

Learners answer multiple-choice questions. StudyQuest records selected answers, score, pass state, XP and explanations. A learner may ask the AI to explain a question belonging to their quest. If explanation generation is unavailable, the saved question explanation remains available.

XP rewards correct answers and first-time level passes. Completing the final level for the first time earns a completion bonus. Repeated attempts award reduced XP and eventually no answer XP. A perfect score unlocks the Perfect Score badge. Activity updates the current and best streak.

Mechanic	Purpose
XP	Rewards correct answers and first-time passes; supports progress and leaderboard ordering.
Levels	Break learning into ordered steps, with a title, summary, reward and pass threshold.
Streak	Recognizes consecutive activity days and records the learner best.
Badges	Marks milestones such as a perfect level score.
Leaderboard	Ranks active learners by XP, then completed certificate count.

AI configuration
Gemini/Gemma is primary; Groq is the quest-generation fallback. Provider keys stay server-side. Content quality and availability depend on the providers and current usage limits.

4. Achievements, certificates and sharing

Completing every level issues a certificate tied to the learner and quest. It is designed to be attractive and independently verifiable.

The certificate area supports search by quest title, topic or verification code; score threshold filters; and sorting. Downloads are generated as PDF. The certificate design includes dynamic app name and logo, learner name and photo when available, quest title and level or difficulty, average score, issue date and a QR code for public verification.

A unique verification code identifies the record. The QR code opens a public page where visitors can verify the credential without signing in. Sharing actions include copying the link, LinkedIn, WhatsApp and the device share sheet where supported.

The public landing page includes a certificate preview. Learners can see certificates in their own area and download PDFs.

> **Level on the certificate**
> The certificate includes the completed quest difficulty or level designation, such as Beginner, Intermediate or Advanced, alongside the score.

5. Profiles, notifications and invitations

These features support learner identity, timely updates and community growth.

Profiles include email, name, bio, avatar, location, school, website, notification preference, XP, streaks and account activity. Profile images accept upload or URL. A learner can delete their account after confirming their password; related learning data is removed according to database relationships.

Notifications belong to the signed-in user. Opening one marks it read before following its safe local destination. Learners can filter read and unread items, mark all as read, clear one, clear selected, or clear all. The header can show the unread count.

Invitations are sent to an email address. Ordinary learners can invite learners; the owner can issue elevated invitations. The system checks for existing accounts and active invitations. Invitations expire after seven days. Acceptance creates the account and signs the new learner in.

> **Email delivery**
> Welcome, invitation, password-reset and contact messages use branded HTML email templates. Delivery needs SMTP credentials. Development falls back to the console email backend; invitation links can be copied from the interface.

6. Public site, dynamic identity and contact

A central site settings record provides the identity shared across the interface.

Dynamic settings include application name, tagline, description, logo, authentication-page image, contact email, phone, address, support URL, accent color, footer text and registration status. Shared templates use these values in the header, sidebar, footer and authentication pages. The animated loader uses the brand identity as well.

The home page supports up to three active hero slides. Each has an eyebrow, headline, body, call-to-action and destination, order, active state, and optional text, image or video media. Staff can manage slides, upload media or enter a URL. The responsive design aims to show the core hero content in the initial viewport.

The contact form validates the sender and saves a ContactMessage. It attempts to email the configured contact address and records whether delivery succeeded. Staff can filter messages by status and mark them new, in progress or resolved. A honeypot field helps reject basic automated submissions.

7. Owner and staff administration

The custom admin area offers operational controls. Server-side permissions guard routes; the most sensitive operations are owner-only.

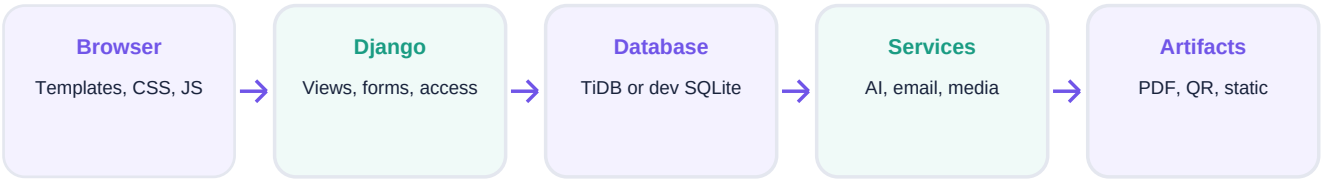
Admin area	Capabilities
Overview	Counts for users, quests and certificates, plus recent users.
Users	Search accounts. Owner can edit fields, role and status, set a new password, update profile image and delete an account with related records.
Site settings	Change name, logo, tagline, colors, contact details, footer, registration and authentication artwork.
Hero slides	Create, edit, activate, reorder and remove up to three slides; use text, image or video media.
Integrations	Set Cloudinary, AI and email credentials. Blank secret fields preserve existing values. Secrets are encrypted using the Django secret key.
Contact inbox	Review messages, see email-delivery status and update follow-up state.

Protect the owner account
The owner cannot remove their own staff, superuser or active status through the user-edit form, and cannot delete their own account in the admin user list.

Staff can manage operational content such as slides and contact messages. The superuser can change roles, edit arbitrary users and manage service credentials.

8. How the application is built

StudyQuest is a server-rendered Django project. Django owns routing, account sessions, forms, permissions, data access and HTML rendering.



Django app	Responsibility
accounts	Custom email account, sign-in, signup, invitations, profiles, password recovery and account deletion.
quests	Quest, level, question and attempt records; dashboards, generation, scoring, XP and leaderboard.
certificates	PDF creation, sharing data and public verification.
notifications	Personal notifications, read state, clear actions and unread API.
core	Landing, branding and integration settings, contact, hero slides, media and admin pages.
config	Django settings, middleware, database selection, URL routing and WSGI entry point.

The shared template base provides branding, fixed header and sidebar navigation, footer and feedback toasts. Guest and signed-in layouts show appropriate links. Responsive CSS adapts the workspace and forms for mobile. WhiteNoise serves static files in the Django deployment.

9. Data model in plain language

The data is organized around ownership: a learner owns quests and attempts; quests contain levels; levels contain questions; finishing a quest can produce a certificate.

Record	Stores
User	Unique email, login, profile fields, XP, streaks, preferences and Django permissions.
Invitation	Email, invited role, token, sender, expiry and acceptance time.
Quest	Owner, title, topic, difficulty, study source, image, status, provider and timestamps.
Level / Question	Ordered learning step, reward and pass threshold; prompt, choices, correct answer and explanation.
Attempt	Learner answers, score, pass result, XP and timestamp.
Badge / UserBadge	Badge definition and the date earned.
Certificate	Learner, completed quest, average score, public code, issue date and optional PDF URL.
Notification	Learner, message, destination, type, read state and creation date.
Site / Integration settings	Brand, media, contact configuration, provider credentials and update time.
HeroSlide / ContactMessage	Ordered public slide and media; submitted contact details, message, status and email flag.

Deleting a user cascades through owned quests, attempts, certificates and notifications. A quest deletion removes its levels and question history. Certificate verification uses its unique UUID code.

10. Security, privacy and media

The app uses Django session, CSRF, password and permission protections. Production safety also depends on deployment configuration.

- Use a strong private DJANGO_SECRET_KEY. It signs sessions and encrypts stored integration secrets; changing it can make existing encrypted values unreadable.
- Keep database, AI, SMTP and Cloudinary secrets in protected environment variables or the owner-only integrations page. Do not commit .env.
- Production uses MySQL-compatible TiDB with TLS certificate and identity verification. SQLite is selected only for local DEBUG development when TiDB variables are absent.
- Use HTTPS in production. Secure session and CSRF cookies, proxy HTTPS detection, host/origin allowlists, HSTS and content-type protections are enabled when DEBUG is off.
- Use Cloudinary for persistent uploads because Vercel function storage is temporary. Media fields accept upload or URL where supported.
- AI requests may include a topic or learner study text. Avoid sensitive personal information in source notes and review provider privacy terms.
- Rotate credentials and separate preview from production database and provider keys.

Media input options
Upload or enter a URL for supported media such as profile images, quest covers, branding and hero slides. Study notes can be typed, uploaded as PDF or text, or read from a source URL.

11. Run locally

The development setup uses SQLite automatically when DEBUG is on and TiDB variables are absent.

Step	Action
1. Prepare Python	Install Python 3.12, create and activate a virtual environment.
2. Install packages	Run pip install -r requirements.txt from the project root.
3. Configure	Copy .env.example to .env, set a development DJANGO_SECRET_KEY and DEBUG=1. Add provider settings only as needed.
4. Prepare database	Run python manage.py migrate. Use python manage.py createsuperuser to create a local owner.
5. Start	Run python manage.py runserver and visit http://localhost:8000.
6. Check	Run python manage.py test; use python manage.py check and makemigrations --check before releases.

Environment settings

Setting group	Purpose
DJANGO_SECRET_KEY, DEBUG	Django signing and environment behavior.
TIDB_HOST, TIDB_PORT, TIDB_USER, TIDB_PASSWORD, TIDB_DATABASE	TiDB connection; required values select TiDB.
GEMINI_API_KEY, GEMMA_MODEL; GROQ_API_KEY, GROQ_MODEL	AI generation and fallback.
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	Persistent media.
EMAIL_HOST_USER, EMAIL_HOST_PASSWORD	Gmail SMTP.
SITE_URL, ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS	Canonical links and accepted web origins.

See .env.example for exact variable names. Never commit production secrets. Provider credentials can also be maintained in the owner-only settings page.

12. Production deployment

The repository targets Vercel Services. The Django app service receives public requests through the catch-all rewrite. The legacy browser prototype is an internal Node service and is not public unless routing is changed.

Configure production environment variables in Vercel: a strong Django key, DEBUG=0, TiDB connection, allowed hosts and CSRF origins, canonical SITE_URL and service credentials. For a public demo, make sure Vercel Deployment Protection does not require visitors to sign in to Vercel. Vercel access protection is separate from StudyQuest account sign-in.

Provision and test the production database before migrations. Apply Django migrations before the first live release. Use README commands as a starting point and verify them against the current Vercel CLI and project configuration. Smoke-test public landing and assets, sign-in, a protected learner page, database access, and certificate verification.

Configure Cloudinary for persistent uploads. Vercel function files are temporary. Test SMTP from production because contact and password reset rely on outbound email. Ensure service names and routes in vercel.json match the Vercel project.

> **Deploy checklist**
> 1. Configure secrets in Vercel. 2. Provision and test TiDB. 3. Run migrations. 4. Configure Cloudinary and email. 5. Set public deployment access. 6. Deploy and test guest, learner and owner workflows.

13. Routes and feature map

The Django URL configuration is the source of truth if paths change.

Path family	Key actions
/	Public landing page and hero.
/contact/	Submit a message.
/accounts/	Signup, login/logout, profile, invitations, password recovery/change and deletion.
/dashboard/	Authenticated learning summary.
/quests/	List, create, view, delete, play levels, submit answers, results and explanations.
/certificates/	View/filter certificates and download PDFs.
/verify//	Public certificate verification.
/notifications/	Filter, read, open or clear notifications.
/leaderboard/	Learner rankings.
/admin/	Staff/owner overview, users, settings, slides, integrations and contact inbox.

Interface behavior

The shared layout includes fixed header/sidebar navigation, a mobile sidebar toggle, global feedback toasts, and an animated learning-themed loader that uses the dynamic app identity. Landing, sign-in and recovery pages, contact, learner workspace and management pages share the visual system.

14. Maintenance and current limits

These routines help keep the platform dependable after its first deployment.

- Commit Django migrations with model changes, then apply them in the same environment serving the release.
- Back up TiDB using provider backup and restore tools, and test a restore process.
- Monitor AI usage and rate limits. If both providers fail, show the learner a useful retry path and preserve their input.
- Monitor email delivery. Contact messages remain stored even when email notification fails.
- Check Cloudinary when media previews fail; externally hosted URLs must be reachable by the browser.
- Use the owner admin carefully for role changes and deletions because associated data can be removed.
- After provider, database or deployment changes, smoke-test home, signup, login, quest generation, an attempt, notification open, certificate download and verification, contact, and admin.

What external setup controls

Some behaviors depend on live service configuration: Vercel public-access settings, database credentials and reachability, AI model availability and limits, SMTP authentication and sending limits, and Cloudinary. This guide omits all credentials. Keep them in protected settings and rotate keys that have been exposed in messages or documents.

> **A note on completeness**
> This guide describes the repository implementation and intended behavior. Provider plans, models, hosting options and terms may change; check current external settings during deployment.

StudyQuest

Learn with purpose. Grow with every quest.

Prepared from the source code, README, Django models, views, routes and settings. October 2026.